

Test Automation Proof of Architecture — 4-Week Plan

Three-Track POC (Traditional vs. AI-Assisted vs. AI-Native) with a Verified Path to 100% Automation

Context: ~20 desktop/web/POS/Unix applications, ~1,500 existing manual test scenarios.

End goal: 100% test automation of the existing scenario set across all applications. **What**

this 4-week POA delivers: a fact-based, POC-backed tool/architecture recommendation;

proof that every blocker category standing between today and 100% is solvable

(demonstrated, not asserted); and the **verification machinery** (coverage contract, traceability matrix, dashboard, gates) that will prove 100% as the follow-on program executes.

The 100% goal — definition, path, and verification

1. The Coverage Contract (agreed and signed in Week 1)

"100%" is only verifiable once it is defined. On Day 1-3, agree in writing:

- **The denominator:** all active scenarios after a documented cleanup — duplicates merged, obsolete scenarios retired *with client sign-off and an audit trail* (denominator cleanup must be transparent, never a trick to hit the number).
- **What counts as "automated" (Definition of Automated):** a scenario counts only if it (a) executes unattended in the CI pipeline, (b) is stable — e.g., $\geq 95\%$ pass consistency over its last 10 scheduled runs absent a real defect, (c) has its result synced automatically to the test management tool, and (d) validates the same intent as the original manual scenario (traceable, reviewed).
- **Accepted automation forms:** fully scripted UI automation; AI-native intent-based execution; **UI automation with emulated peripherals** (virtual card reader, printer, scanner); **API/service-level automation** where it verifies the same business rule; semi-automated only if explicitly listed and time-boxed for conversion.

2. Gap-closure strategies — how "not automatable" becomes automatable

Every scenario in the estate is assigned to exactly one closure strategy. Nothing is allowed to sit in an undefined "manual forever" pile.

Blocker category	Closure strategy	Proven in POC?
Standard UI flows (web/desktop)	Track A/B scripted automation	Yes — Days 9–12
Legacy desktop / terminal apps with poor object recognition	Track C vision/agentic execution; terminal emulation (SSH/screen-scrape)	Yes — Day 13
Hardware peripherals (card reader, receipt printer, barcode scanner, cash drawer)	Peripheral emulation/simulation: ESC/POS printer emulators, virtual card-reader/payment- simulator (PSP test harness), scanner input injection (HID/keyboard-wedge simulation)	Yes — Day 13–14 (dedicated spike)
Unavailable / rate-limited integrations (payment gateways, third parties)	Service virtualization / API mocking (e.g., WireMock, Mountebank, vendor sandbox)	Yes — Day 14
Scenarios blocked by app design (no test IDs, dynamic controls, no test env hooks)	Testability backlog raised with dev teams (stable locators, test APIs, seams); interim: vision-AI recognition	Feasibility assessed; backlog delivered Day 17
Scenarios containing human-judgment steps (exploratory checks, visual acceptability)	Scenario refactoring: split into an automatable deterministic part + visual-AI validation for the judgment part; re-word acceptance criteria to be assertable	Sample refactors in POC
Genuinely physical one- offs (e.g., physically swiping a damaged card)	Re-specify at the closest automatable layer (emulated equivalent) per the Coverage Contract	Contract decision, Day 3

3. The verification mechanism — how 100% is proven, continuously

- **Automation Traceability Matrix (ATM):** the single source of truth. Every scenario ID (all ~1,500) → closure strategy → automated test ID → owning track/tool → last CI run status → stability score. Started Day 3, handed over live at Day 20.
- **Coverage dashboard:** generated automatically from CI results + test management sync (never hand-maintained). Reports per-app and overall: % automated per the Definition of Automated, % in progress, % blocked with named blocker and owner.
- **Coverage gates:** roadmap milestones (e.g., 60% at 6 months, 85% at 12, 100% at 18) each verified by the dashboard plus a **sample-based independent audit** — randomly

sample N automated scenarios per app, confirm they run unattended, are stable, and genuinely verify the original intent.

- **Regression protection for the number:** CI quality gate fails the build if coverage regresses (tests deleted/quarantined without an approved ATM change); quarantined/flaky tests count as **not automated** until restored — no silent erosion of the 100%.
- **What the POA itself verifies (Week 4):** that every blocker category has a POC-demonstrated or contractually agreed closure path, that the ATM covers 100% of the denominator with a strategy per scenario, and that the measurement pipeline works end-to-end on the POC apps.

The three tracks (definitions the whole engagement hangs on)

Track	Definition	Test artifact	Example stacks
A — Traditional	Deterministic scripted automation; human-authored locators & code; no AI	Script + locator repository	Selenium/Playwright, Robot Framework, WinAppDriver/FlaUI, SSHLibrary, UFT One (classic)
B — AI-Assisted	Architecture still scripted & deterministic; AI helps humans build and maintain it — LLM conversion of manual scenarios, copilot authoring, self-healing locators	Script (AI-generated/AI-repaired, human-owned & reviewed)	Playwright/Robot + LLM generation (Claude/Copilot), Healenium, Testim, mabl, Katalon AI, Tosca Copilot
C — AI-Native	AI is the execution engine; tests expressed as intent, interpreted against the UI at runtime; no locator repo or conventional script; recognition/healing intrinsic	Intent / NL test definition	testRigor, Functionize, Eggplant/KaneAI agentic modes, computer-use agents, vision-based runtime recognition

Capability classification (keep explicit in the report): LLM scenario conversion, copilot authoring, and bolt-on self-healing = B; vision-AI runtime recognition and agentic intent execution = C; visual AI validation = borderline (AI-native validation usable inside B); AI test-data generation and flaky triage = track-agnostic accelerators.

Guiding principles

- Evaluate against *this* estate; POC apps must include the hardest 20% (POS with peripherals, legacy desktop, terminal).
 - Same scenarios across all three tracks — every claim becomes a measured delta.
 - **No scenario without a closure strategy** — the ATM must account for 100% of the denominator by Day 17.
 - Working hypothesis to test openly: B is the regression workhorse; C is deployed surgically where only it can reach. Confirm or overturn with data.
 - Measure honestly: false positives, run-to-run determinism, and per-run cost alongside success rates.
-

POC hypotheses & metrics

#	Hypothesis	Metric	Track
1	LLM generation cuts authoring 40–70% on the backlog	Authoring time B vs. A; % generated scripts runnable with < 15 min fix-up	B vs. A
2	Self-healing cuts maintenance 50–80% after UI change	Broken tests, healing success rate, human fix time	B vs. A
3	AI-native unlocks apps traditional tools cannot see	% of "hard" scenarios automatable only in Track C	C
4	AI-native intent tests are regression-reliable	Pass consistency over 20 runs; step variance; false pass/fail rate	C
5	Peripheral emulation closes the hardware blocker	# of hardware-dependent POS scenarios automated end-to-end with emulated devices	closure spike
6	Service virtualization closes the integration blocker	# of integration-blocked scenarios automated against virtualized services	closure spike
7	Per-run economics differ materially	Cost per scenario run per track at 1,500 × release-cadence scale	A/B/C
8	Verification pipeline works	ATM auto-updated from CI results on POC apps; dashboard renders true coverage	all

Risks to probe on Track C: hallucinated assertions, non-determinism, auditability, PCI/data privacy for POS screens sent to cloud AI, token cost at regression volume, lock-in.

Week 1 — Discovery, Coverage Contract & Estate Analysis

Day 1 — Kickoff, access & the 100% conversation

- Kickoff: scope, decision-makers, three-track methodology buy-in.
- **Table the Coverage Contract:** propose the Definition of Automated, denominator rules, and accepted automation forms (emulated peripherals, API-level equivalence). Getting this negotiated from Day 1 is what makes "100%" a verifiable engineering target instead of a slogan.
- Request access: apps, environments, test management, CI/CD, source control.
- AI governance: cloud AI permitted? PCI constraints on POS screens? Self-hosted LLM needed?

Day 2 — Application inventory

- Landscape matrix for all ~20 apps: type, technology, hosting, release cadence, criticality, cross-app E2E flows.
- Flag Track C candidates (poor object recognition) and **peripheral/integration dependencies per app** (feeds the closure-strategy mapping).

Day 3 — Scenario analysis & ATM v0

- Export all ~1,500 scenarios; run the **denominator cleanup** (duplicates, obsolete) with client sign-off and audit trail.
- Build **ATM v0**: every remaining scenario gets app, type, frequency, blocker category (if any), and a *provisional closure strategy* from the table above. From this day on, the ATM is the single source of truth for the 100% goal.
- Assess documentation quality: % of backlog LLM-convertible as written.

Day 4 — Stakeholder interviews & constraints

- QA/dev/ops interviews: pain points, past automation failures, skill profile, budget.
- **Dev-team testability conversation:** appetite and process for a testability backlog (test IDs, hooks, seams) — a prerequisite for closing the app-design blocker category.
- Audit/compliance expectations for AI-generated and agentic tests.

Day 5 — Evaluation framework & tool longlist

- Weighted criteria (example): technology coverage 20%, authoring effort 15%, maintainability incl. self-healing 15%, AI maturity & accuracy 10%, determinism & auditability 8%, CI/CD & reporting 8%, skill fit 8%, TCO incl. AI consumption 10%, vendor support 6%.
 - Longlist mapped to tracks (A: Playwright/Selenium, Robot + FlaUI/WinAppDriver/SSHLibrary, UFT One, Ranorex, TestComplete; B: A-stack + LLM generation + Healenium, or Testim/mabl/Katalon AI/Tosca Copilot; C: testRigor, Functionize, Eggplant/KaneAI agentic, computer-use agents; Applitools as visual layer).
 - **Closure-spike shopping list:** candidate peripheral emulators (ESC/POS printer emulator, payment/PSP simulator, HID scanner injection) and service-virtualization tools (WireMock/Mountebank/vendor sandboxes).
 - Client sign-off: criteria, weights, three-track methodology, **and the Coverage Contract**.
-

Week 2 — Shortlisting & POC Setup

Day 6 — Shortlist & POC scope

- One stack per track (B is often A + AI layers — mirrors the realistic adoption path).
- 3-4 representative apps: modern web, legacy desktop, POS, Unix/terminal.
- Scenario selection: 10-12 common scenarios for all three tracks, 3-4 hard scenarios for the Track C unlock test, **and at least one scenario from every blocker category** (hardware-dependent, integration-blocked, judgment-step) so each closure strategy gets POC evidence.

Day 7 — Environments, licenses & closure-spike setup

- Trials/installs; POC machines/VMs; verify each stack can see each app type.
- **Install and smoke-test the closure tooling:** printer emulator, payment simulator, scanner input injection, service-virtualization server.
- Enable AI features; LLM access within security rules; masked/synthetic data policy for anything sent to cloud AI.

Day 8 — POC design, scorecard & measurement pipeline

- Lock the scorecard (hypotheses table) per track per scenario.

- **Stand up the verification pipeline skeleton:** ATM as structured data (sheet/DB), CI results feeding it automatically, first dashboard render — prove the measurement machinery on day one of building, not at the end.
- Framework skeletons per track.

Day 9 — Build: Track A baseline

- Automate the common scenarios traditionally; log effort meticulously (the denominator for every improvement claim).

Day 10 — Track A finish + Track B conversion pilot + checkpoint

- Finish Track A; 10+ stability runs.
 - Track B pilot: feed 20–30 real manual scenarios into LLM generation; measure % runnable with < 15 min fix-up.
 - **Weekly checkpoint:** estate results, ATM v1 (every scenario now has a blocker category and closure strategy), baseline numbers, conversion-rate data. Walk the client through *how* 100% will be reached and verified — strategy by strategy.
-

Week 3 — Three-Track Build & Closure Spikes

Days 11–12 — Build: Track B (AI-assisted)

- Same scenario set via LLM/copilot authoring into the scripted framework; wire in self-healing.
- Capture authoring-time deltas vs. Track A; count human review effort on generated code as real cost.

Day 13 — Build: Track C (AI-native) + hardware closure spike

- Same scenarios as natural-language intent; run the reserved hard POS/terminal scenarios.
- Run Track C scenarios 10–20 times today — determinism is the make-or-break metric.
- **Hardware spike:** automate 2–3 peripheral-dependent POS scenarios end-to-end using the printer emulator / payment simulator / scanner injection. This is the evidence that the biggest "impossible" bucket is closable.

Day 14 — Integration closure spike + CI/CD operability, all tracks

- **Integration spike:** automate 1–2 integration-blocked scenarios against virtualized services.

- All three tracks into the client pipeline: triggered runs, parallelism, reports, results synced to test management — **and flowing into the ATM/dashboard automatically** (hypothesis 8).
- Auditability: healing events logged (B), agent decisions explainable (C).

Day 15 — Maintenance experiment + full-run soak

- Deliberately change UI elements/screens; measure per track: tests broken, healing rate (B), agent adaptation (C), human fix time, false-heal rate.
 - 20+ full-suite runs; final flakiness and false-positive numbers; verify the dashboard reflects reality throughout.
 - Licensing quotes incl. AI consumption; cost per regression cycle per track at full scale.
-

Week 4 — Analysis, Path-to-100% Plan & Recommendation

Day 16 — Consolidate POC data

- Weighted scorecard per track, measured data only.
- Backlog extrapolation: person-months to convert the full denominator via A vs. B vs. C, at measured reliability and run cost.

Day 17 — The Path-to-100% plan & ROI

- **Finalize the ATM: 100% of the denominator mapped**, each scenario to exactly one closure strategy, with per-strategy effort estimate, owner type (QA/dev/vendor), and dependency (e.g., testability backlog items for dev teams — delivered today as a prioritized list).
- Coverage statement in closure buckets: scripted (A/B) / AI-native-only (C) / via peripheral emulation / via service virtualization / pending testability change / refactor-then-automate — **summing to 100%, with zero unassigned scenarios**.
- ROI per track and for the tiered combination; sensitivity to release cadence; cost of the closure enablers (emulators, virtualization, testability work) included honestly.

Day 18 — Tiered recommendation, milestones & verification governance

- Recommendation traced to evidence (expected shape, confirmed or overturned by your data: B as workhorse, C surgical, A only where AI is prohibited).
- **Roadmap with verified coverage milestones:** e.g., 60% at 6 months, 85% at 12, 100% at 18 — each gate = dashboard number + independent sample audit per the Coverage Contract.

- Verification governance handed to the client: Definition of Automated enforced in CI, coverage-regression quality gate, quarantine policy (flaky = not automated), ATM change control, monthly coverage reporting, audit cadence.
- AI governance: review gates for AI-generated assertions, self-healing audit policy, agentic-run approval, data masking.

Day 19 — Report & presentation prep

- POA report: estate analysis, Coverage Contract, three-track results per hypothesis, closure-spike evidence, Path-to-100% plan, risks, tiered recommendation, cost model, roadmap.
- Executive deck: lead with four numbers — authoring reduction (B vs. A), maintenance reduction (B/C vs. A), coverage unlocked by AI-native (C), and **"100% of scenarios mapped to a proven closure strategy."**
- Dry run / peer review.

Day 20 — Presentation & handover

- Present exec-first; live moment if possible: break a locator (A fails, B heals, C doesn't care) and run a POS sale against the emulated printer.
- Hand over: POC assets, scorecards, raw data, **the live ATM and dashboard**, testability backlog, AI-generated-script samples with review notes.
- Agree next steps: procurement, pilot team, conversion program kickoff against the milestone plan.

Deliverables checklist

1. **Signed Coverage Contract** (Definition of Automated, denominator rules, accepted forms)
2. Application landscape & technology matrix (Track C suitability + peripheral/integration dependencies)
3. Cleaned, signed-off scenario denominator with audit trail
4. **Automation Traceability Matrix — 100% of scenarios mapped to a closure strategy**, live-fed from CI
5. Coverage dashboard (auto-generated) + verification governance pack (gates, audit method, quarantine policy)
6. Three-track POC assets + head-to-head scorecard per hypothesis, incl. closure-spike evidence (peripheral emulation, service virtualization)
7. Testability backlog for dev teams (prioritized)

8. Path-to-100% plan with milestones, effort, owners; 3-year TCO & ROI per track and tiered combination
9. Tiered recommendation report + executive presentation

Key risks to manage

- **Coverage Contract not signed early** — without an agreed Definition of Automated, "100%" is unverifiable and the engagement's success criterion is undefined. Escalate if not signed by end of Week 1.
- **Denominator gaming** — scenario cleanup must be transparent and signed off; the audit trail protects your credibility.
- **Access/license delays** — raise Day 1, escalate Day 3; AI features and emulator procurement often need vendor enablement.
- **Hardware closure underestimated** — peripheral emulation is the highest-risk spike; if an emulator doesn't exist for a device, the fallback (vendor test harness, hardware-in-the-loop rig) must be costed in the Path-to-100% plan, not hand-waved.
- **Testability dependency on dev teams** — part of the path to 100% is app changes you don't control; the backlog needs an executive owner from Day 18.
- **AI over-claiming** — report false-positive and determinism numbers; count human review time on generated tests.
- **Data privacy** — masked/synthetic data to cloud AI from Day 7; PCI check for POS screens.
- **Silent coverage erosion post-100%** — the CI regression gate and quarantine policy are what keep 100% true after it's first achieved.